



UNIVERSITÀ
DEGLI STUDI
FIRENZE

UNIVERSITÀ DI FIRENZE

SCUOLA DI INGEGNERIA

Corso di Laurea Triennale in Ingegneria
Elettronica e delle Telecomunicazioni

**Sviluppo e test
di un sistema di telemetria
per veicoli a guida autonoma**

Relatori:

Prof. Michele Basso

Prof. Enrico Boni

Correlatore:

Dott. Luca Bigazzi

Candidato:

Lorenzo Tucci

ANNO ACCADEMICO 2022/2023

Indice

1	Introduzione	1
2	Modello trasmissione Teensy	4
2.1	Configurazione UART	4
2.2	Struttura dei dati inviati	4
3	Modello Simulink per l'elaborazione dei dati	6
3.1	Blocchi Simulink utilizzati nel modello	7
3.2	Modello finale e configurazione dei blocchi	8
3.2.1	Funzione per il calcolo della Checksum	11
4	Test e verifica della robustezza del modello	13
4.1	Caso standard	13
4.2	Caso interruzione di trasmissione	14
4.3	Caso pacchetto non valido	15
5	Conclusioni	16

Capitolo 1

Introduzione



Figura 1.1: Rover

Il Dipartimento di Ingegneria dell'Informazione (**DINFO**) dell'Università degli Studi di Firenze ha sviluppato un progetto che consiste nella costruzione di un *rover* radiocomandato in scala 1:8, riportato in figura 1.1, sul quale è stato implementato un sistema elettronico per la guida autonoma.

Questo progetto si distingue per l'implementazione di un'architettura elettronica complessa, che opera su due livelli computazionali distinti: il primo, di alto livello, è

composto dal sistema di telerilevamento per la guida autonoma ed è gestito da una scheda di sviluppo Jetson Orin; il secondo, di basso livello, è costituito da una scheda Teensy 4.1 che si interfaccia con motore e sterzo sia a livello di comandi sia a livello di sensoristica. La scheda Teensy è affiancata da una scheda Arduino, la quale si occupa della gestione dei dati provenienti dalla Jetson e dalla Teensy stessa.

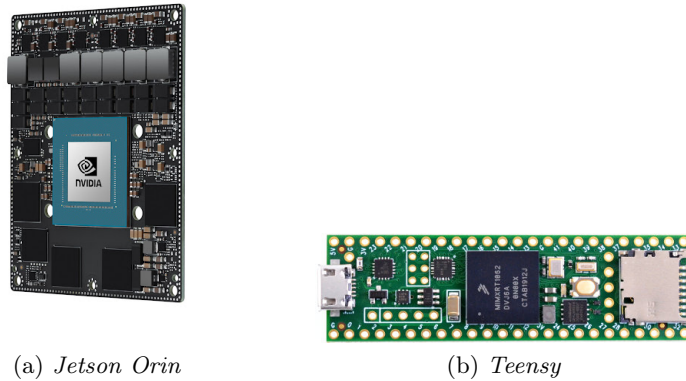


Figura 1.2: Schede di sviluppo del rover

I dispositivi di rete usano un sistema di regole, cioè un protocollo di comunicazione, per trasmettere informazioni tramite tecnologie fisiche o *wireless*. Nel nostro caso la comunicazione della Teensy con la Jetson e con Arduino avviene attraverso il protocollo seriale UART.

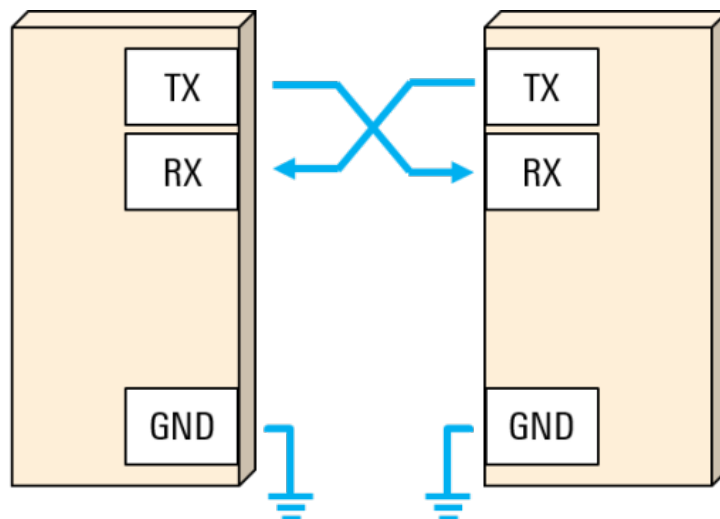


Figura 1.3: Connessione seriale UART tra due dispositivi

L'UART o *Universal Asynchronous Receiver Transmitter* è un protocollo di comunicazione seriale tra due dispositivi elettronici che svolge un ruolo importante

nelle applicazioni *embedded*. È un protocollo molto semplice che utilizza due fili che mettono in connessione il dispositivo trasmettitore e quello ricevitore e un filo di massa a comune (Figura 1.3).

Un grande vantaggio del protocollo UART è il fatto di essere asincrono: di conseguenza il *master* e lo *slave* non condividono un segnale di *clock* comune; questa proprietà semplifica molto il protocollo, anche se impone che i due dispositivi rispettino alcuni requisiti. Essi devono trasmettere i dati alla stessa velocità, cioè con lo stesso *Baud Rate*, e devono essere configurati con gli stessi parametri per avere la stessa struttura del *frame*. Il Baud Rate rappresenta la frequenza di simbolo, mentre la struttura di *frame* la lunghezza della parola trasmessa. Inoltre, questo protocollo permette di implementare una comunicazione sia monodirezionale (**Single-Duplex**) sia bidirezionale, in cui invia dati un dispositivo alla volta (**Half-Duplex**) o entrambi i dispositivi inviano dati contemporaneamente (**Full-Duplex**).

Il mio elaborato di tesi si colloca all'interno del progetto descritto in precedenza, focalizzandosi sullo sviluppo e il test di un sistema di telemetria tra la Teensy e l'Arduino. Per sviluppare tale sistema ho creato un modello Simulink, software per modellazione, simulazione e analisi di sistemi dinamici, sviluppato dalla compagnia statunitense MathWorks. Questo strumento mi ha permesso di creare un modello accurato del sistema di comunicazione, facilitando lo sviluppo e il *testing* delle soluzioni proposte.

Capitolo 2

Modello trasmissione Teensy

2.1 Configurazione UART

La scheda Teensy, programmata in C++, contiene una parte di codice per la configurazione della porta seriale.

I parametri, solitamente associati alla trasmissione dati seriale asincrona, sono definiti attraverso il campo *c_flag* che descrive il controllo hardware del terminale. Essi sono ad esempio il bit di parità e la dimensione dei bit di dati, ovvero la lunghezza della parola trasmessa.

Nel nostro caso è stato impostato:

- **460800** - Baud rate;
- **CS8** - dimensione bit dati 8 bit;
- **PARODD** - bit di parità dispari;
- **CLOCAL** - ignora le linee di stato del modem;
- **CREAD** - abilita il ricevitore;

2.2 Struttura dei dati inviati

Nelle reti di computer e nelle telecomunicazioni, un *frame*, noto anche come "data frame", è un'unità di trasmissione di dati digitali. Un frame è composto tipicamente da caratteristiche di sincronizzazione costituite da una sequenza di bit o simboli, che indicano al ricevitore l'inizio e la fine dei dati del *payload* all'interno del flusso di simboli o bit che riceve. Essi sono noti come *Header* e *Footer*. La Teensy è incaricata di inviare un pacchetto di dati di dimensione pari a 32 byte.

Tale pacchetto è codificato in un dato strutturato *struct toHighLevel* i cui i campi

sono i dati da inviare. I pacchetti sono configurati come riportato nella tabella sottostante:

Header	Payload	Checksum	Footer
4 byte	20 byte	4 byte	4 byte

L'Header è un *int32*, cioè un valore intero, noto al ricevitore, codificato su 32 bit, che permette di riconoscere l'inizio del pacchetto a cui è stato assegnato il valore di 453. Analogamente il Footer è un *int32* che definisce la fine del messaggio ed ha valore pari a 761.

In sintesi, il pacchetto è composto da un Header e un Footer che delimitano il Payload, che rappresenta il carico informativo utile, mentre il Checksum è un *int32* che serve per verificare che i dati inviati siano corretti e che, ad esempio, non ci sia stata perdita di informazione durante la trasmissione.

La logica che assegna il valore alla variabile Checksum è definita dalla funzione *computeChecksumTx* che somma tutti i valori delle variabili contenute nel payload e attribuisce il totale all'intero Checksum.

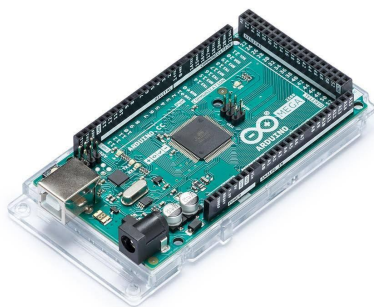
Questo tipo di struttura dati è comune nelle comunicazioni seriali e nelle reti informatiche, dove è importante garantire l'accuratezza e l'integrità delle informazioni scambiate tra i dispositivi. La chiara definizione di Header, Payload, Checksum e Footer aiuta a organizzare e interpretare correttamente i dati trasmessi.

Capitolo 3

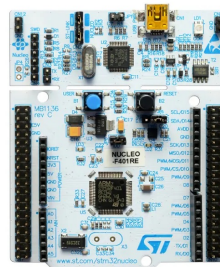
Modello Simulink per l'elaborazione dei dati

I dati che vengono trasmessi dalla scheda Teensy alla scheda Arduino devono essere riordinati e verificati prima di essere utilizzati. Per realizzare questo compito è stato implementato un modello Simulink che permette non solo di gestire la ricezione dei pacchetti, ma in un futuro anche di creare un sistema di controllo per il rover, che ne gestisce la velocità e lo sterzo.

Il pacchetto di Simulink *Simulink Support Package for Arduino Hardware* ha reso possibile la realizzazione del modello, poichè fornisce la tecnologia di progettazione Model-Based per creare sistemi *embedded* su Arduino. Le simulazioni e i test sono stati effettuati utilizzando un Arduino Mega 2560 e un NUCLEO-F410RB di STM32.



(a) *Arduino Mega 2560*



(b) *NUCLEO-F410RB*

Figura 3.1: Schede utilizzate per i test

La realizzazione del modello è stata suddivisa in due fasi. Durante la prima è stato creato un prototipo fittizio del *data frame* testato sull'Arduino attraverso un'interfaccia di loopback. Questo tipo di interfaccia è frequentemente usato per testare il funzionamento di un sistema: nel nostro caso, ovvero in quello di un sistema di comunicazione, il test di *loopback* consiste nel collegare con un cavo la porta di trasmissione e la porta di ricezione seriale, facendo in modo che il messaggio inviato

torni indietro allo stesso dispositivo da cui è partito. La seconda fase ha comportato invece l'utilizzo di una scheda STM32 (in 3.1b) che simulasse ciò che trasmette la Teensy.

3.1 Blocchi Simulink utilizzati nel modello

In questo paragrafo vengono elencati i blocchi utilizzati nel modello e il loro ruolo.

Il primo blocco, utilizzato per i test in loopback, è il ***Serial Transmit***, rappresentato in figura 3.2, che prende in ingresso un array di dimensione $[N \times 1]$ o $[1 \times N]$ e richiede di selezionare quale è, a livello hardware, la porta seriale di trasmissione.

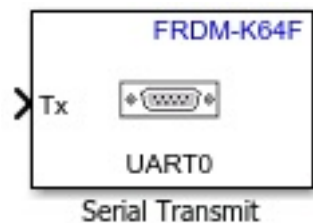


Figura 3.2: Blocco Serial Transmit

Il ***Serial Receive***, rappresentato in figura 3.3, rappresenta la porta seriale di ricezione. I parametri da specificare sono la porta hardware associata, il tipo di dato che presenta in uscita dal blocco, la dimensione in byte dei dati da ricevere, che è limitata ad un massimo di 32 byte, e il *Sample Time*. Quest'ultimo parametro specifica l'intervallo di tempo con cui la porta di ricezione viene letta per acquisire i dati in arrivo.

Il blocco presenta in uscita un segnale contenente i dati ricevuti ed un segnale *Status* che assume il valore 1 quando i dati sono pronti per essere letti sulla porta di ricezione, indicando che una nuova trasmissione è stata ricevuta ed è pronta per essere elaborata. Se non ci sono dati disponibili, il segnale di *Status* assume il valore 0, indicando che la porta di ricezione è in attesa di nuovi dati.

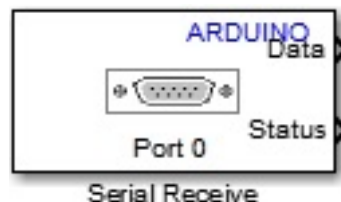


Figura 3.3: Blocco Serial Receive

Durante la fase di ricerca e di testing, sono stati utilizzati i blocchi ***Scope*** e ***To Workspace***. Quest'ultimo, permettendo di salvare nel Workspace di MatLab i dati

di un segnale, è stato molto utile per effettuare il *debug*.

Il blocco più importante, riportato in figura 3.4, è il **Protocol Decoder** che, opportunamente configurato, si occupa di decodificare il messaggio ricevuto.

Nei parametri vengono indicati Header e Footer, Checksum e relativa dimensione, Payload, e infine la dimensione, in byte, di tutto il messaggio da decodificare. In ingresso prende un flusso di byte di tipo *uint8*, cioè valori codificati come interi senza segno su 8 bit. In uscita vengono mandati tanti segnali quanti sono i campi del Payload indicati nei parametri del blocco con il tipo specificato; inoltre escono due segnali booleani *IsValid* e *IsNew* che assumono valore 1 rispettivamente se il pacchetto è valido e se è disponibile un nuovo gruppo di dati decodificati.

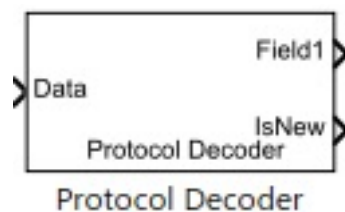


Figura 3.4: Blocco Protocol Decoder

Infine gli ultimi blocchi utilizzati sono una serie di **Display** che permettono di visualizzare i dati in uscita dal blocco Protocol Decoder.

3.2 Modello finale e configurazione dei blocchi

Il primo passo della realizzazione del modello è stata la corretta configurazione della porta seriale. Dopo aver selezionato, nelle impostazioni hardware del modello Simulink, la scheda Arduino giusta, è possibile accedere al pannello per settare il Baud Rate, la lunghezza della parola e il tipo di bit di parità, come si può vedere per la porta seriale 3 in figura 3.5. È importante tenere presente che Simulink intende il bit di parità escluso dai bit della parola, quindi per impostare i parametri come nella Teensy, dove la UART è configurata con 8 bit di parola, 1 bit di parità dispari e 1 bit di stop, è necessario selezionare l'opzione (**SERIAL_8O1**).

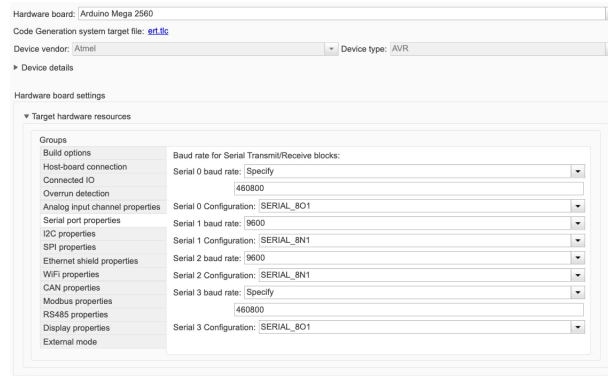


Figura 3.5: Configurazione porta seriale su Simulink

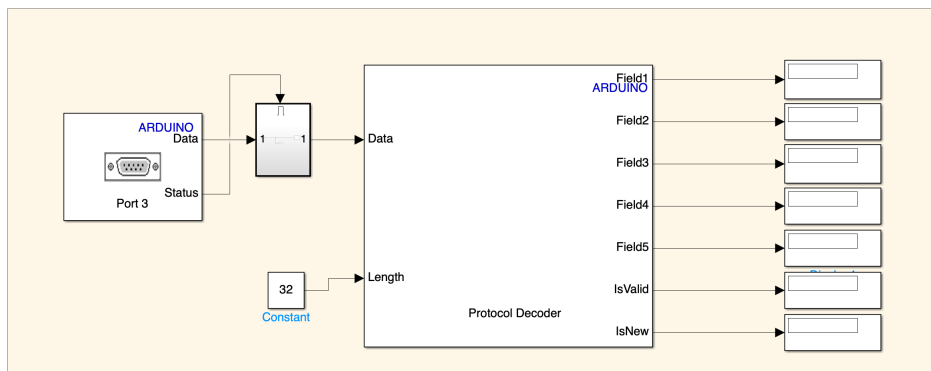


Figura 3.6: Modello finale

Il modello finale, riportato in figura 3.6, presenta come primo blocco il *Serial Receive* che riceve il pacchetto di dati e lo manda ad un *Enabled Subsystem*.

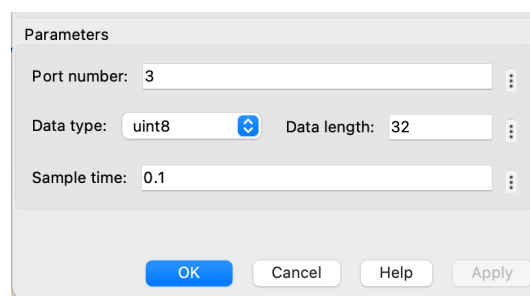


Figura 3.7: Parametri blocco Serial Receive

Come si vede dai parametri riportati in figura 3.7, il tipo di dati di uscita è **uint8**, poiché tale segnale sarà l'input del blocco *Protocol Decoder*, il quale richiede che il segnale di ingresso sia di questo tipo.

Essendo i dati trasmessi interi a 32 bit, in uscita dal blocco ogni valore sarà scomposto nei 4 byte, a partire dal bit meno significativo. Ad esempio, l'Header che ha valore 453, sarà passato come [197 1 0 0].

È molto importante tenere conto della scomposizione del valore intero in 4 byte separati, poiché questo influisce sulla specificazione di Header, Footer e logica di Checksum nel **Protocol Decoder**. Dovendo essere ancora scelta la frequenza di trasmissione dei dati da parte della scheda Teensy, i grafici e i test sono stati eseguiti ipotizzandone una di 10 Hz. Se la lettura della porta seriale avviene ogni 100 ms si può vedere che il segnale Status è costante ad un valore logico alto, mentre se avviene ogni 50 ms lo Status è un'onda quadra di periodo 100 ms con duty cycle del 50%. Il comportamento di quest'ultimo caso è ben visibile dal grafico in figura 3.8.

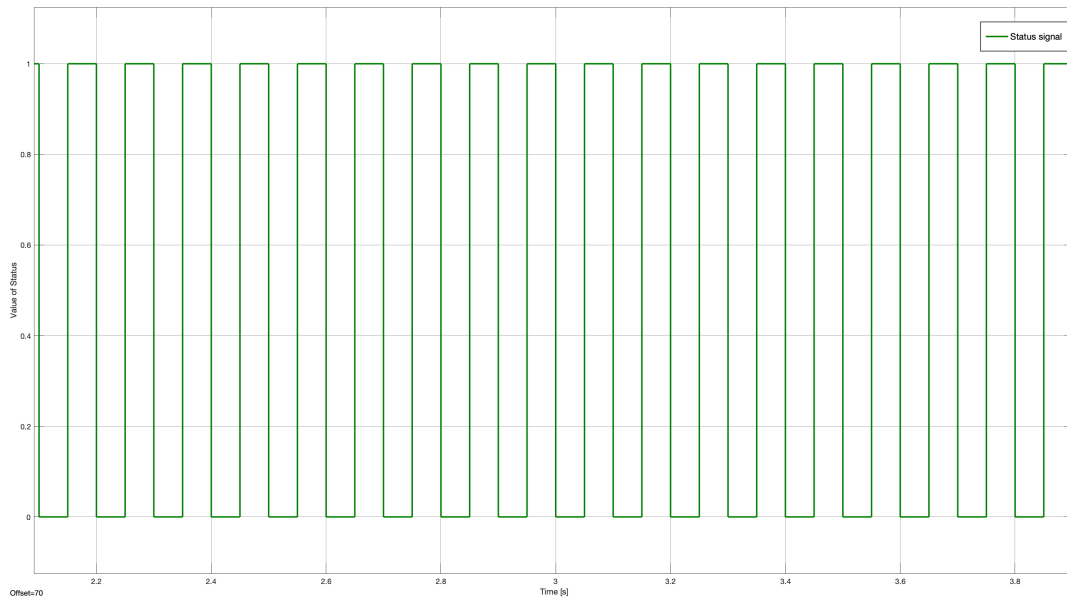


Figura 3.8: Segnale *Status*

L'*Enabled Subsystem* è un sottosistema che viene eseguito quando un ingresso booleano assume un valore *True*. L'input del sottosistema è il segnale *Status*, il quale garantisce che, al Protocol Decoder, arrivino soltanto pacchetti di dati ricevuti e non, per esempio, gruppi di zeri dovuti al fatto che la porta seriale viene letta anche quando i dati non sono disponibili. In sintesi, il suo obiettivo è quello di filtrare i pacchetti di dati ricevuti.

Il **Protocol Decoder** è il componente fondamentale che assicura che i dati ricevuti siano correttamente interpretati dal sistema, garantendo così un funzionamento affidabile e coerente dell'intero sistema di comunicazione o elaborazione dati. Header e Footer, nel blocco noto come Terminator, devono essere riportati come singoli valori di tipo *uint8* o come array di dati sempre di tipo *uint8*. Ricevendo in ingresso uno *stream* di byte di questo tipo, risulta chiaro il motivo per cui il blocco impone questa richiesta.

Essendo l'Header e il Footer definiti come *int32* nello struct di partenza, è stato necessario riportarli, nei campi dei parametri del blocco, come array di quattro valori *uint8*. Questi quattro numeri corrispondono alla divisione dell'intero a 32 bit in 4

byte, letti a partire dal bit meno significativo.

L'Header, al quale viene assegnato il numero **453**, viene indicato come **[197 1 0 0]**; mentre al Terminator, che ha valore pari a **761**, l'array **[249 2 0 0]**.

Dal momento in cui è nota la lunghezza del frame, è possibile selezionare la casella "*Data input length available*" che abilita un secondo ingresso nel quale specificare tale misura. A questo input è collegato un blocco **Constant** con valore 32, *uint8* come tipo di dato di uscita e *Inf* come *Sample Time*.

Il decoder dedica una sezione alla specifica suddivisione dei dati del Payload, dove è possibile indicare il nome della variabile associata al dato, il tipo di uscita, la lunghezza e l'ordine dei bit. Nel progetto, per semplicità, sono stati considerati cinque campi di tipo **int32**, anche se nelle conclusioni sarà trattata una possibile suddivisione del Payload utile per l'applicazione all'interno del progetto.

L'ultimo parametro configurabile all'interno del blocco è la logica di Checksum, ovvero un metodo o un algoritmo che determina se il pacchetto è valido o da scartare. L'uso della Checksum è facoltativo, e dunque è necessario attivare tale opzione. Dopo aver selezionato "*Custom algorithm*" viene richiesto di specificarne la dimensione in byte e un file MatLab (con estensione .m) nel quale viene implementata la funzione adibita al calcolo della Checksum. Il file deve necessariamente condividere lo stesso *path* del modello.

3.2.1 Funzione per il calcolo della Checksum

In questa sottosezione viene brevemente spiegato il codice MatLab della function che si occupa di calcolare e verificare la Checksum.

```
function isValid2 = validateData(packet, checksumBytes)
isValid2= false;
A= cell(4,1);
for k= 1:4
    A{k}=dec2bin(checksumBytes(k),8);
end
checksumvec = horzcat(A{4},A{3},A{2},A{1});
checkSum_value = bin2dec(checksumvec);
E = cell(20,1);
for i = 1: 20
    E{i} = dec2bin(packet(i),8);
end
C = cell(5,1);
C{1} = horzcat(E{4},E{3},E{2},E{1});
C{2} = horzcat(E{8},E{7},E{6},E{5});
C{3} = horzcat(E{12},E{11},E{10},E{9});
C{4} = horzcat(E{16},E{15},E{14},E{13});
C{5} = horzcat(E{20},E{19}, E{18},E{17});
sumdata= 0;
D = cell(5,1);
for j=1:5
    D{j}= bin2dec(C{j});
    sumdata = sumdata+ D{j};
end

if checkSum_value == sumdata
    isValid2 = true;
end
```

Figura 3.9: Function per il calcolo della Checksum

Come si può notare nel codice riportato in figura 3.9 la funzione prende in ingresso due parametri e restituisce in uscita un valore booleano che viene inizializzato a false.

I parametri in ingresso sono due array che contengono i byte del Payload e i byte della Checksum. Nella sezione 2.2 viene spiegato come funziona il calcolo dell'intero checksum; un procedimento analogo è quello che viene replicato su Arduino. Come detto in precedenza, i dati entrano entrano nel *Protocol Decoder* come uno *stream* di *uint8*; perciò non è possibile effettuare una semplice somma e confrontarla con l'intero passato nel messaggio, ma è necessario ricostruire i valori interi di ogni singolo dato prima di eseguire tale operazione.

Per ricostruire ogni intero, si deve trasformare ogni byte nel corrispettivo valore binario grazie alla funzione *dec2bin()*, la quale restituisce come tipo un *char*; dopodiché, a gruppi di quattro, si concatenano orizzontalmente con *horzcat()* questi valori in modo da ottenere il valore binario corrispondente all'intero a 32 bit.

Si può notare come nei parametri di *dec2bin()* venga inserito il numero 8; ciò è molto importante poichè assicura che il dato, anche se rappresentabile su pochi bit, venga espresso in 8 bit evitando successivamente possibili problemi nella concatenazione. Dopo aver riportato in decimale i valori espressi in binario con *bin2dec()*, questi vengono sommati, escludendo Header e Footer, e confrontati con l'intero Checksum, che è stato trattato analogamente.

Se il confronto avviene con successo, significa che il pacchetto di dati è corretto e al booleano viene assegnato il valore true, cosicché in uscita dal blocco vedremo i dati decodificati e il segnale *Is Valid* con valore pari a 1.

Capitolo 4

Test e verifica della robustezza del modello

In questo capitolo vengono presentati i test e i conseguenti risultati che sono stati eseguiti per verificare il corretto funzionamento del modello. Non essendo stata ancora decisa la frequenza di trasmissione dei dati da parte della Teensy, per i seguenti test ne è stata ipotizzata una di 10 Hz.

4.1 Caso standard

Il primo caso analizzato mira a verificare il corretto funzionamento del modello in una situazione standard, cioè in cui non ci sono interruzioni nella trasmissione e i dati sono sempre validi.

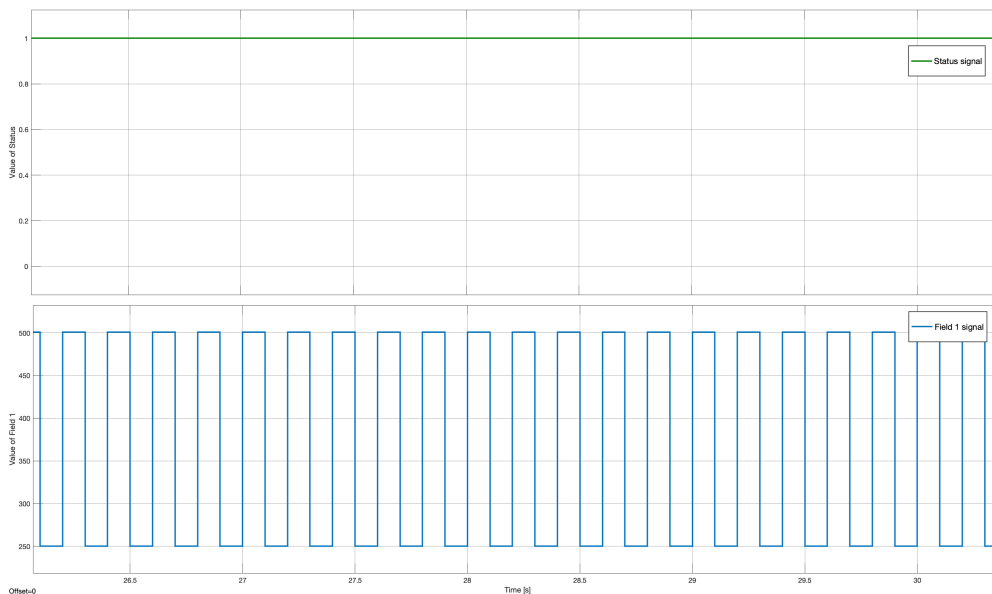


Figura 4.1: Segnale *Status* e Segnale *Field 1*

La lettura della porta seriale, così come l'invio dei dati dalla Teensy, avviene ogni 100 ms quindi, come si vede in figura 4.1, il segnale *Status* assume un valore logico alto in modo costante. Il secondo segnale raffigurato è il primo dato del pacchetto ed è stato misurato in uscita dal blocco **Protocol Decoder**. In questo caso si può notare che il dato varia correttamente tra due valori ogni 100 ms. Il fatto che vari tra **501** e **250** è causato dalla struttura dati inviata attraverso la **STM32-NUCLEO**; infatti sono stati creati due *struct data* che vengono inviati alternativamente ogni 100 ms.

4.2 Caso interruzione di trasmissione

Il caso analizzato in questa sezione si propone di studiare il comportamento del modello nel caso in cui avvenga un'interruzione della trasmissione. Questa situazione può essere dovuta, ad esempio, alla disconnessione del connettore come allo spegnimento del dispositivo trasmettente o anche ad un *Reset* effettuato su quest'ultimo.

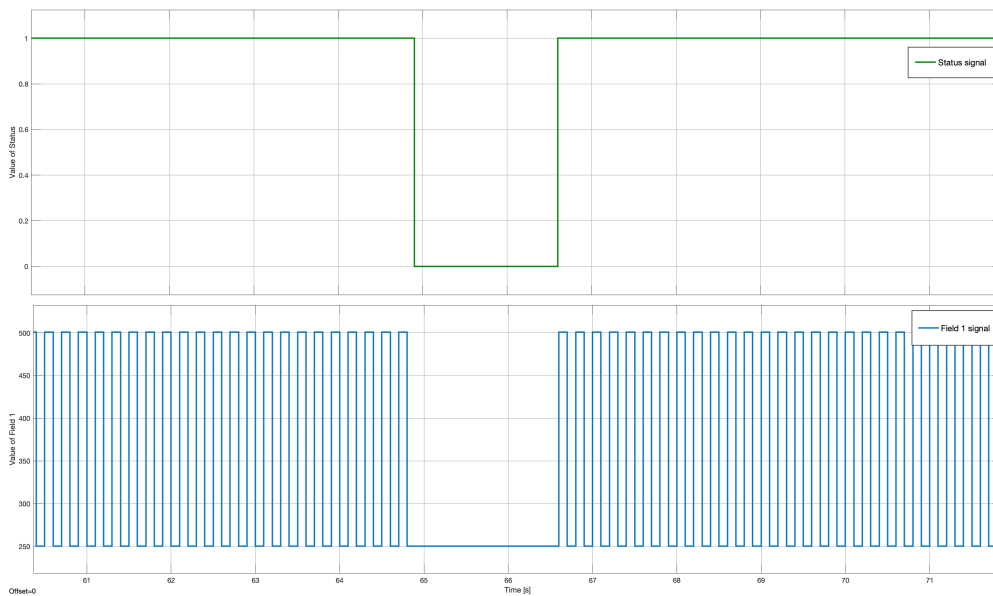


Figura 4.2: Segnale *Status* e Segnale *Field 1*

Nella figura 4.2 è riportato il comportamento del segnale *Status* e del segnale *Field 1* nel momento in cui viene effettuata un'azione di *Reset* sul dispositivo incaricato di inviare i dati.

È possibile notare come lo *Status* passa immediatamente da un valore logico alto ad uno basso per poi ritornare a 1 soltanto quando viene ristabilita la connessione. Dal secondo segnale si può invece osservare come si comporta il **Protocol Decoder**; infatti questo blocco mantiene in uscita l'ultimo dato valido poichè l'*Enabled Subsystem* non permette il passaggio di dati non consistenti lasciando in uscita l'ultimo

pacchetto inviato.

4.3 Caso pacchetto non valido

L'ultimo test effettuato è stato quello per verificare il corretto funzionamento della Checksum. Per farlo, sulla STM32-NUCLEO, sono stati creati due pacchetti di dati: uno con l'intero Checksum corretto e uno con l'intero Checksum sbagliato. L'invio alternato dei due *struct* avviene ogni 100 ms, come nei test precedenti.

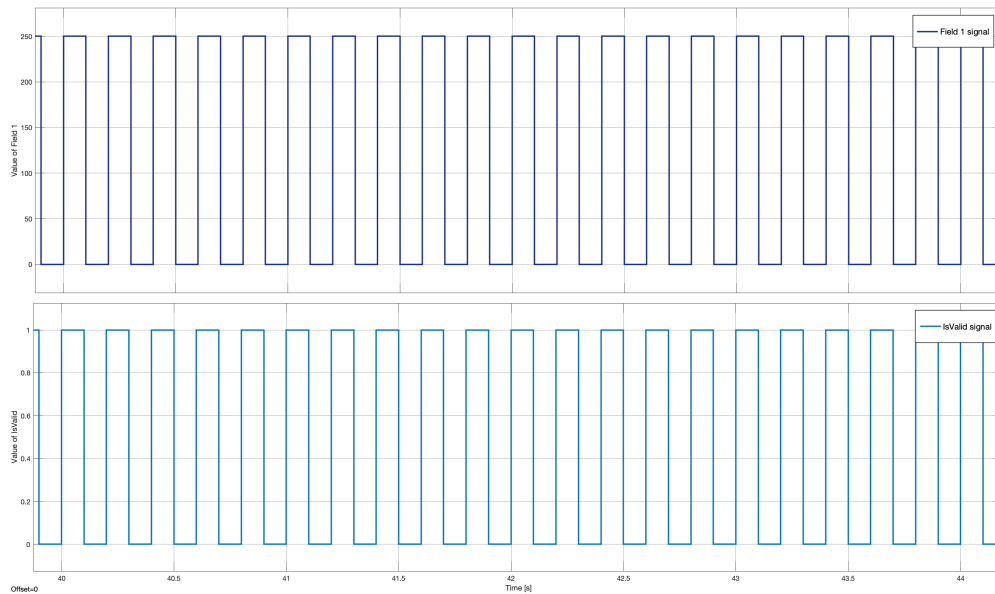


Figura 4.3: Segnale *Field 1* e Segnale *IsValid*

Nel grafico, rappresentato in figura 4.3, si nota come il segnale *IsValid* assuma un valore logico basso ogni 100 ms, cioè ogni volta che viene inviato il pacchetto sbagliato. Quando *IsValid* va a 0, in uscita dal *Protocol Decoder* i campi del Payload assumeranno valore nullo, come si nota dal comportamento del segnale *Field 1* nel grafico di figura 4.3.

A causa di questo comportamento, possiamo concludere che è necessario utilizzare, a valle del *Protocol Decoder*, un *Enabled Subsystem* regolato dal segnale *IsValid* affinché possano essere utilizzati soltanto dati consistenti validi.

Capitolo 5

Conclusioni

Al termine di questo progetto di tesi è stato possibile trarre delle conclusioni riguardo alla corretta ricezione e decodifica dei dati e riguardo alle future implementazioni del progetto.

Come è visibile dai test trattati nel capitolo precedente, è possibile affermare che il modello è in grado di riconoscere e decodificare correttamente i pacchetti di dati inviati dalla Teensy.

Risulta un problema invece la gestione dei dati se vengono inviati più di 32 byte alla volta, poichè la bufferizzazione sulla porta seriale di ricezione risulta difficile da implementare. La difficoltà maggiore si rivela comprendere esattamente il funzionamento del blocco ***Serial Receive***, poichè, non essendo open source, non si è in grado di capire come manda in uscita lo stream di dati ricevuto.

Alternativamente, è consigliabile ottimizzare il pacchetto inviato dalla Teensy, indicando un payload adeguato ai 32 byte di dimensione.

La struttura finale del messaggio può essere riorganizzata come indicato nella tabella sottostante:

Header	Payload	Checksum	Footer
2 byte	26 byte	2 byte	2 byte

L' Header e il Footer possono essere codificati in due *int16*, occupando quindi 4 byte, invece degli otto precedenti.

Anche la Checksum può essere codificata su 2 byte come *int16* o, in alternativa, può essere implementata con una logica XOR o CRC. Quest'ultima garantirebbe inoltre un controllo più affidabile sul controllo del pacchetto.

Rimanendo quindi 26 byte disponibili, il Payload può contenere tredici campi di tipo *int16*, nei quali saranno codificati i dati dei sensori e dei comandi utili per un futuro sistema di controllo.

La Teensy invierebbe:

- cinque *int16*, che rappresentano le quattro velocità delle ruote e la velocità media;
- sei *int16*, ovvero i valori dell'accelerazione e del giroscopio sui tre assi X,Y,Z;
- due *int16* per codificare i comandi, cioè sterzo e gas.

Il progetto si concluderà con la creazione di un sistema di controllo che, utilizzando i dati inviati dalla Teensy, possa correggere attivamente l'assetto e la velocità del rover per evitare che si verifichi un fenomeno di slittamento e/o sbandamento.

Bibliografia

- [1] 2022, The MathWorks Inc., MATLAB version: 9.13.0 (R2022b), The MathWorks Inc., Natick, Massachusetts, United States, <https://www.mathworks.com>
- [2] 2022, The MathWorks Inc., Simulink Support Package for Arduino Hardware version: 22.2.2 (R2022b), The MathWorks Inc., Natick, Massachusetts, United States, <https://it.mathworks.com/matlabcentral/fileexchange/40312-simulink-support-package-for-arduino-hardware>

Ringraziamenti

Ringrazio la mia famiglia per il costante supporto durante questo percorso e tutti i miei amici per avermi fatto divertire ogni giorno. In particolare un ringraziamento agli *Amici miei*, storici, che dalle Muse mi hanno portato in la isla e a tutti i membri di *GipPino* e di *Va Deciso* con cui ho condiviso tanti bei momenti.

Un ringraziamento anche a tutti i ragazzi del *Francone* senza i quali sicuramente non sarei qua adesso, soprattutto ai *Carissimi amici* che hanno reso questi anni di università veramente unici e pieni di risate.

Infine un grazie anche alla mia ragazza che è stata un grande punto di riferimento per me.

Vi voglio bene a tutti.